

Clase 14 – Actividad 1

¿Cómo funciona el backtracking en este problema?

El backtracking en este problema intenta colocar reinas una por una en cada fila, verificando si la posición es segura con `isSafe()`. Si una colocación es válida, el algoritmo avanza a la siguiente fila. Si en algún punto no hay movimientos válidos, el algoritmo "retrocede" (backtrack), eliminando la última reina colocada y probando la siguiente posible ubicación en la fila anterior.

¿Qué pasa cuando el algoritmo encuentra una solución?

Cuando el algoritmo coloca exitosamente N reinas sin conflictos, la condición `if (row == N)` se cumple y retorna `true`. Esto hace que todas las llamadas recursivas previas también devuelvan `true`, finalizando el proceso.

¿Qué ocurre cuando no puede colocar más reinas?

Si en una fila no hay ninguna posición segura, el algoritmo retorna `false`, provocando un retroceso a la fila anterior para intentar otra opción. Si en la primera fila tampoco hay alternativas, el problema no tiene solución.

¿Qué sucede en el código cuando el algoritmo "retrocede"? ¿Cómo se visualiza en Python Tutor?

El retroceso ocurre cuando `solve(board, row + 1)` devuelve `false`. En ese caso, la reina colocada en `board[row][col]` se elimina (`board[row][col] = 0`), y el algoritmo prueba la siguiente columna en la misma fila.

En Python Tutor, se vería como una pila de llamadas recursivas donde la ejecución retrocede a niveles anteriores para probar una nueva posibilidad.

¿Qué modificaciones harías para aumentar N a 8? ¿Cómo crees que cambiaría el tiempo de ejecución?

Para cambiar N a 8, basta con modificar `private static final int N = 30;` a `private static final int N = 8;`.

El tiempo de ejecución disminuiría considerablemente en comparación con N = 30, ya que el número de combinaciones a evaluar es menor. Sin embargo, al aumentar N más allá de 30, el tiempo de ejecución crecería exponencialmente debido a la cantidad de configuraciones posibles.

¿Por qué el método isSafe es crucial en este algoritmo?

`isSafe` es fundamental porque verifica que una reina no sea atacada antes de colocarla. Sin esta validación, el algoritmo intentaría todas las combinaciones sin restricciones, lo que haría imposible encontrar una solución válida.